Universidade Federal de Viçosa – Campus Florestal
Projeto Integrador 2025 - Arquitetura de Software
Equipe 2

Prova de Conceito

Pingo: Uma Aventura na Rede

O presente documento descreve a Prova de Conceito (PoC) do jogo “Pingo: Uma Aventura na Rede”, idealizado e desenvolvido pela Equipe 2 no âmbito do Projeto Integrador 2025.

Versão: 1.0

Data: 10/09/2025

Autor(a): Ana Carolina Fernandes, Arquiteta de Software

Sumário

1. Visão Geral..... 3
2. Descrição Técnica..... 4
2.1. Estrutura do Projeto e Tecnologias Web (Front-end).....5
2.2. Ambiente de Desenvolvimento (Back-end).....6
2.3. Gerenciamento de Credenciais e Segurança..... 7
2.4. Hospedagem e Comunicação.....7
3. Versão das Tecnologias..... 8
4. Implementação das Tecnologias.....8
4.1. Front-End..... 9
4.2. Back-end..... 10
5. Execução..... 11
6. Links Úteis.....12

1. Visão Geral

O presente documento consiste na arquitetura utilizada para a construção da Prova de Conceito (PoC) do projeto "Pingo: Uma Aventura na Rede". Foram definidas as tecnologias, frameworks e bibliotecas principais utilizadas durante o desenvolvimento, detalhando o processo de decisão e a interação entre os componentes.

A arquitetura foi projetada para suportar uma aplicação web interativa, conforme o ecossistema da imagem abaixo.

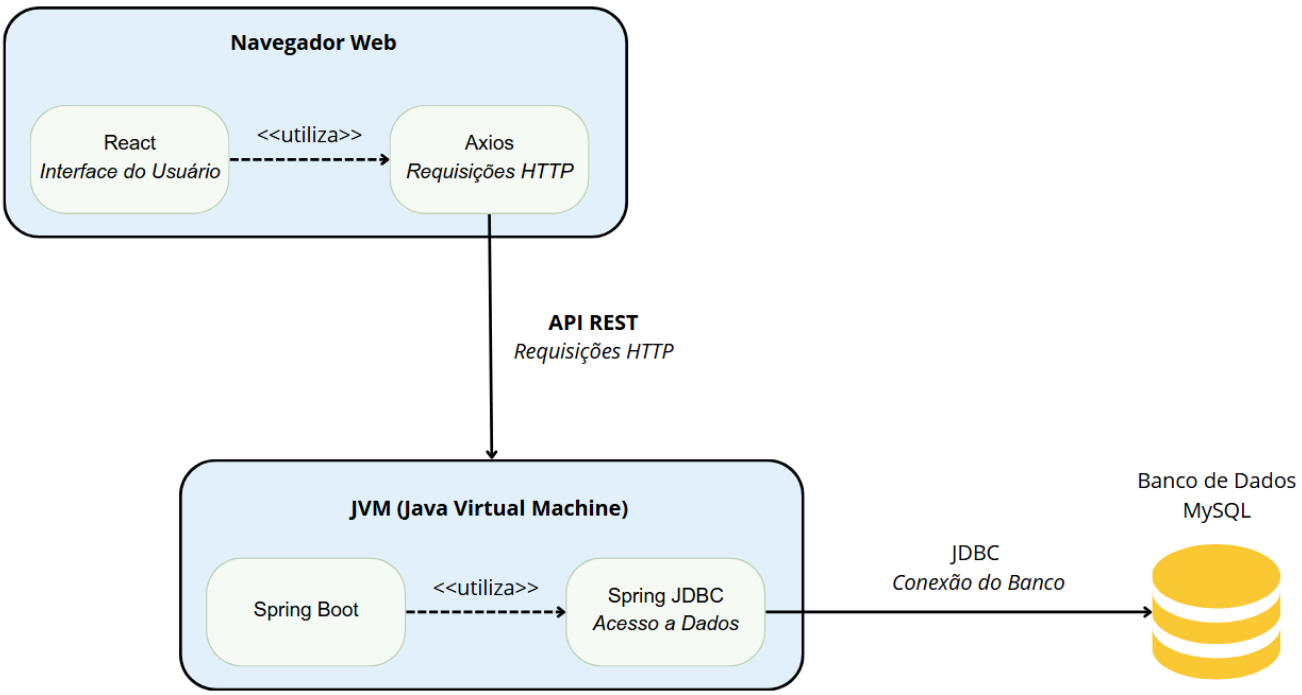


Figura 1 - Visão Geral da Arquitetura Proposta

- *Front-end*:
  - *React*: Biblioteca para construir a interface do usuário.
  - *Vite*: Ferramenta de build para um desenvolvimento rápido e moderno.
  - *Axios*: Biblioteca para comunicação com o back-end via API REST.
- *Back-end*:
  - *Spring Boot*: Framework para a construção da API REST.
  - *Spring JDBC*: Ferramenta para acesso manual ao banco de dados via SQL.
  - *Gradle*: Gerenciador de dependências e do projeto.
- Banco de Dados:

- *MySQL*: Sistema de gerenciamento de banco de dados relacional.

A arquitetura do *back-end* foi projetada para atender a dois critérios principais: as exigências disciplinares do Projeto Integrador e a facilidade de aprendizado para os desenvolvedores. A exigência do uso da linguagem ***Java*** foi atendida com a adoção do ***Spring Boot***, um framework que acelera o desenvolvimento de *APIs REST*. Da mesma forma, a exigência do uso do ***MySQL*** fez com que, ao invés de uma abstração completa com ORM, fosse feita a implementação de um DAO (Data Access Object) manual com ***Spring JDBC***. Esta escolha servirá para incentivar a escrita direta de *queries SQL* por parte dos juniores.

Para o *front-end*, foi mantida a decisão de utilizar ***React*** pela sua popularidade e pela imensa quantidade de recursos de aprendizado disponíveis. Visando uma maior facilidade, a ferramenta de *build* utilizada foi o ***Vite***. A comunicação com o *back-end* é realizada por meio de uma *API REST*, com a biblioteca ***Axios*** gerenciando as requisições HTTP, escolhida por sua simplicidade e sintaxe intuitiva.

2. Descrição Técnica

Nesta seção, serão citados alguns pontos importantes relacionados à integração das tecnologias, ao ambiente de desenvolvimento e à estrutura do projeto.

Para criar a estrutura inicial do projeto *front-end* com o *Vite*, é necessário ter o *Node.js* e o *npm* instalados, e utilizar o seguinte comando:

```
None
npm create vite@latest pingo-vite -- --template react
```

- *npm create vite@latest*: Utiliza a versão mais recente do *Vite* para criar um novo projeto;
- *pingo-vite*: Define o nome da pasta onde o projeto será criado;
- *-- --template react*: É uma instrução passada diretamente ao *Vite* para que ele use o modelo pré-configurado para *React*, já com a estrutura de pastas e arquivos básicos (index.html, main.jsx, etc.)

Para executar o projeto *front-end* pela primeira vez,. O primeiro passo é instalar as dependências listadas no arquivo *package.json*. Para isso, navega-se até a pasta raiz do *front-end* (pingo-vite) e executa-se o comando:

```
None
npm install
```

Este comando baixa todas as bibliotecas necessárias (como *React* e *Axios*) para a pasta local *node_modules*. Após a instalação, o servidor de desenvolvimento é iniciado com o comando:

```
None
npm run dev
```

Isso iniciará o servidor do *Vite*, e a aplicação estará acessível no navegador, geralmente no endereço *http://localhost:5173*.

2.1. Estrutura do Projeto e Tecnologias Web (*Front-end*)

O *front-end* é organizado em componentes, o que vem da biblioteca *React*. Como visto na implementação, cada parte da interface (*Login.jsx*, *ListaFases.jsx*, *Fase04.jsx*) é um componente que funciona por conta própria, cuidando tanto da sua aparência quanto de suas funcionalidades.

```javascript
JavaScript
import './App.css';

// Declaração do componente como uma função que recebe 'props' (propriedades)
const Login = ({ onLoginSuccess }) => {
    // Toda a lógica de estado e funções do login ficam aqui dentro

    return (
        // A aparência do Componente é definida aqui
    );
};

// O Componente é exportado para poder ser usado em outros lugares
export default Login;
```

Esses componentes são escritos em JSX, que permite escrever uma sintaxe semelhante ao HTML diretamente no código.

```JavaScript
// Exemplo utilizando o mesmo arquivo do código anterior (Login.jsx)
return (
    <div className="login-container">
        <h2>Login do Aluno</h2>
        <form onSubmit={handleSubmit} className="login-form">
            <input
                type="text"
                value={apelido}
                onChange={e => setApelido(e.target.value)}
                placeholder="Apelido"
            />
            <button type="submit">Entrar</button>
        </form>
    </div>
);
```

É possível fazer alterações visuais por meio de arquivos CSS (*App.css*), que são importados pelos componentes para definir cores, fontes e layouts.

```CSS
.login-container {
    background-color: rgba(255, 255, 255, 0.7);
    padding: 40px 50px;
    border-radius: 25px;
    box-shadow: 0 10px 20px rgba(0, 0, 0, 0.2);
    backdrop-filter: blur(5px);
    border: 3px solid white;
    max-width: 450px;
    width: 100%;
}
```

2.2. Ambiente de Desenvolvimento (Back-end)

A criação do projeto *back-end* foi feita utilizando *Spring Initializr*, uma ferramenta web que gera a estrutura inicial de um projeto *Spring Boot*, já com as dependências essenciais, como *Spring Web* e *Spring JDBC*. No entanto, é importante ressaltar que o uso do Initializr é uma conveniência, não uma obrigação. É possível criar a estrutura de pastas manualmente e declarar cada dependência diretamente no arquivo de configuração do *Gradle (build.gradle)*.

Porém, a abordagem via Initializr garante a compatibilidade entre as versões das bibliotecas e facilita o aprendizado da equipe.

Independentemente do método de criação, o arquivo de configuração *build.gradle,* é gerado. O *Gradle* é uma ferramenta de automação de build escolhida para o projeto, gerenciando dependências (baixando todas as bibliotecas necessárias a partir da internet) e compilando todo o código *Java*.

Para o desenvolvimento da PoC, foi utilizada a IDE *IntelliJ IDEA*, mas o *Visual Studio Code* é uma alternativa igualmente viável, já que ambos possuem excelente integração com o *Gradle* e o *Spring*.

2.3. Gerenciamento de Credenciais e Segurança

As credenciais de acesso ao banco de dados *MySQL* (URL, usuário e senha) não são escritas diretamente no código *Java*. Elas são armazenadas no arquivo de configuração *src/main/resources/application.properties*. Por segurança, este arquivo nunca deve ser versionado em sistemas de controle como o *Git*, devendo ser incluído no arquivo *.gitignore* do projeto.

Exemplo de configuração no *application.properties*:

```
None
spring.datasource.url=jdbc:mysql://localhost:3306/nome_do_banco
spring.datasource.username=nome_do_usuario
spring.datasource.password=senha
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Durante a Prova de Conceito, a dependência do *Spring Security* foi adicionada para explorar a forma correta de lidar com senhas, utilizando o algoritmo *BCrypt* para *hashing*. Embora a implementação completa de um sistema de segurança não seja um requisito central desta PoC, a inclusão do PasswordEncoder demonstra a prática recomendada para o armazenamento seguro de senhas, além da utilização da ferramenta ser bastante simples.

2.4. Hospedagem e Comunicação

Para esta PoC, tanto o *back-end* quanto o *front-end* foram executados em ambiente de desenvolvimento local (*localhost*). A comunicação entre o *front-end* (servido pelo *Vite*) e o *back-end* (servido pelo *Spring Boot*) ocorre através de uma API REST, com o *front-end* configurado com um *proxy*, que é feito no arquivo *vite.config.js*, instruindo o servidor do *Vite* a redirecionar todas as chamadas feitas para o caminho */api* diretamente para a porta do servidor *back-end* (8080).

```JavaScript
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
    plugins: [react()],
    server: {
        proxy: {
            // Redireciona requisições que começam com /api para o backend
            '/api': {
                target: 'http://localhost:8080',
                changeOrigin: true,
            }
        }
    }
})
```

3. Versão das Tecnologias

As versões abaixo foram utilizadas na Prova de Conceito e não apresentaram problemas de Integração:

1) Tecnologias do Front-end:
    a) React: 19.1.1
    b) Vite: 7.1.2
    c) Axios: 1.11.0

2) Tecnologias do Back-end:
    a) Java: 17 ou 21 (LTS)
    b) Spring Boot: 3.5.5
    c) Spring JDBC: (gerenciado pelo Spring Boot 3.5.5)
    d) Gradle: 8.14.3
3) Banco de Dados:
    a) MySQL: 8.0.40

4. Implementação das Tecnologias

Nesta seção, serão apresentados exemplos de código da PoC que demonstram o uso e a interação entre as tecnologias.

4.1. Front-End

- *React* e *Vite*: Como citado anteriormente, o *React* funciona através da criação de componentes. O *Vite* compila esses componentes e os serve. O arquivo *App.jsx* atua como o componente principal que gerencia o estado e decide qual tela renderizar.

```javascript
function App() {
    const [aluno, setAluno] = useState(null);
    const [faseAtiva, setFaseAtiva] = useState(null);

    if (!aluno) {
        return <Login onLoginSuccess={handleLoginSuccess} />;
    }

    if (faseAtiva) {
        if (faseAtiva.id === 3) {
            return <Fase03 onSair={handleSairDaFase} />;
        }
        if (faseAtiva.id === 4) {
            return <Fase04 onSair={handleSairDaFase} />;
        }
       // ...
    }

    return <ListaFases onFaseSelect={handleFaseSelect} />;
}
```

- *Axios:* Usado para a comunicação com a API REST. A função *handleSubmit* no componente *Login.jsx* é um exemplo prático.

```javascript
const handleSubmit = async (e) => {
    e.preventDefault();
    try {
        const response = await axios.post('/api/login', { apelido, senha });
        onLoginSuccess(response.data);
    } catch (error) {
        console.error("Erro ao fazer login", error);
    }
```

```
};
```

4.2. *Back-end*

- *Spring Boot:* O Spring Boot facilita a criação de *endpoints* REST. A classe *FaseController* expõe as URLs para o frontend consumir os dados das fases.

```java
@RestController
@RequestMapping("/api/fases")
public class FaseController {
    private final FaseService faseService;

    @GetMapping
    public ResponseEntity<List<Fase>> getFases() {
        return ResponseEntity.ok(faseService.listarFases());
    }
}
```

- *Spring* JDBC: Diferente de frameworks ORM como o Hibernate/JPA, o acesso a dados foi feito manualmente para fins de aprendizado, escrevendo *queries* SQL e mapeando os resultados.

```java
@Repository
public class AlunoDAOImpl implements AlunoDAO {
    private final JdbcTemplate jdbcTemplate;

    @Override
    public Optional<Aluno> findByApelido(String apelido) {
        String sql = "SELECT id, apelido, senha FROM aluno WHERE apelido =
?";
        try {
            Aluno aluno = jdbcTemplate.queryForObject(sql, (rs, rowNum) -> {
                Aluno a = new Aluno();
                a.setId(rs.getLong("id"));
                a.setApelido(rs.getString("apelido"));
                a.setSenha(rs.getString("senha"));
                return a;
```

```
            }, apelido);
            return Optional.ofNullable(aluno);
        } catch (EmptyResultDataAccessException e) {
            return Optional.empty();
        }
    }
}
```

5. Execução

Após toda a contextualização, a execução da prova de conceito deve ser feita seguindo os seguintes passos:

1. Criar um banco local *MySQL* e executar o código *sql* abaixo:

```
SQL
CREATE DATABASE pingo_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE pingo_db;

CREATE TABLE aluno (
                       id BIGINT AUTO_INCREMENT PRIMARY KEY,
                       apelido VARCHAR(255) NOT NULL UNIQUE,
                       senha VARCHAR(255) NOT NULL
);

CREATE TABLE fase (
                      id BIGINT PRIMARY KEY,
                      nome VARCHAR(255) NOT NULL,
                      bloqueada BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE ponto_de_rede (
                               id BIGINT PRIMARY KEY,
                               fase_id BIGINT NOT NULL,
                               tipo VARCHAR(50) NOT NULL,
                               pos_x DOUBLE NOT NULL,
                               pos_y DOUBLE NOT NULL,
                               FOREIGN KEY (fase_id) REFERENCES fase(id)
);

INSERT INTO fase (id, nome, bloqueada) VALUES (1, 'Ligando Aparelhos', true);
INSERT INTO fase (id, nome, bloqueada) VALUES (2, 'Enviando Mensagens',
true);
```

```
INSERT INTO fase (id, nome, bloqueada) VALUES (3, 'Definindo o Caminho',
false);
INSERT INTO fase (id, nome, bloqueada) VALUES (4, 'Navegando na Fibra
Óptica', false);

INSERT INTO ponto_de_rede (id, fase_id, tipo, pos_x, pos_y) VALUES
                                                                (1, 3,
'partida', 10, 50),
                                                                (2, 3,
'roteador', 30, 30),
                                                                (3, 3,
'roteador', 30, 70),
                                                                (4, 3,
'destino_errado', 50, 30),
                                                                (5, 3,
'switch', 60, 70),
                                                                (6, 3,
'destino', 80, 70);
```

2. Preencher o arquivo *application.properties* com os dados do banco
3. Iniciar o servidor *back-end*. (Pode ser feito executando o arquivo *PocApplication.java*)
4. Executar o comando abaixo para instalar as dependências (é importante estar na pasta *pingo-vite)*:

```
None
npm install
```

5. Iniciar o *front-end*

```
None
npm run dev
```

6. Acesse o endereço local do front-end: http://localhost:5173/

6. Links Úteis

- React: https://react.dev/

- Vite: https://vitejs.dev/
- Axios: https://axios-http.com/
- Spring Boot: https://spring.io/projects/spring-boot
- Spring JDBC: https://docs.spring.io/spring-framework/docs/current/reference/html/data-access.html#jdbc
- MySQL: https://www.mysql.com/